

Project Report
AI FOR MEDICAL DIAGNOSIS (IMAGE
CLASSIFICATION)
Artificial Intelligence



Submitted By:

[Huzaifa Shahbaz]

[21-CE-009]

[Aqsa Shah]

[21-CE-039]

Submitted to:

Engr. Hareem Khan

Dated:

19th May, 2025

Department of Computer Engineering,
HITEC University, Taxila

Table of Contents

ABSTRACT:	4
1. INTRODUCTION	5
1.1 INTRODUCTION	5
1.2 PROBLEM STATEMENT	5
1.3 OBJECTIVES	5
2.1 METHODOLOGY	7
2.2 STEPS OF METHDOLOGY	7
3. HISTORY OF MEDICAL DIAGNOSIS	9
3.1 EARLY FOUNDATION(1950s-1070s)	9
3.2 INTRODUCTION OF MEDICAL IMAGE ANALYSIS (1980s-1990s)	9
3.3 RISE OF MACHINE LEARNING(1990s-2010s)	9
3.4 DEEP LEARNING REVOLUTION(2012-PRESENT)	9
3.5 RECENT TRENDS AND FUTURE OUTLOOK(2020s-PRESENT)	10
4. CODE OF AI FOR MEDICAL DIAGNOSIS	11
4.1 CODE OF AI FOR MEDICAL DIAGNOSIS	11
5. RESULTS	30
5.1 RESULTS OF AI MEDICAL DIAGNOSIS FOR IMAGE CLASSIFICATION	30
6. CONCLUSION	38
6.1 CONCLUSION	38
References	38

LIST OF FIGURES

Figure 5.1:REUSLTS OF TENSOR FLOW	30
Figure 5.2:RESULTS OF TRAINING ARTIFICAL NEURAL NETWORK	30
Figure 5.3:RESULTS OF ANN EVALUATION	31
Figure 5.4:GRAPH OF CONFUSION MATRIX OF ANN EVALUATION	31
Figure 5.5:CLASSIFICATION OF MEDICAL DIAGNOSIS OF SHAPES OF ANN EVALUATION ..	31
Figure 5.6:GRAPHS OF ANN ACCURACY AND ANN LOSS OF ANN EVALUATION.....	32
Figure 5.7: RESULTS OF TRAINING CONVOLUTIONAL NEURAL NETWORK.....	32
Figure 5.8:RESULTS OF CNN EVALUATION	33
Figure 5.9:GRPAH OF CONFUSION MARTIX OF CNN EVOLUTION	33
Figure 5.10: CLASSIFICATION OF MEDICAL DIAGNOSIS OF SHAPES OF CNN EVALUATION	33
Figure 5.11: GRAPHS OF CNN ACCURACY AND CNN LOSS OF CNN EVALUATION	34
Figure 5.12: RESULTS OF TRAINING BINARY NEURAL NETWORK.....	34
Figure 5.13:RESULT OF BNN EVALUTION	35
Figure 5.14: GRPAH OF CONFUSION MARTIX OF BNN EVOLUTION	35
Figure 5.15: CLASSIFICATION OF MEDICAL DIAGNOSIS OF SHAPES OF BNN EVALUATION	35
Figure 5.16: GRAPHS OF BNN ACCURACY AND BNN LOSS OF BNN EVALUATION	36
Figure 5.17:MODEL COMPARSION OF ANN,CNN AND BNN MODEL	36
Figure 5.18:GRAPH OF BEAM SEARCH ALGORTHIM	36
Figure 5.19:GRPAH OF BEST-FIRST SEARCH ALGORTHIM.....	37

ABSTRACT:

AI is revolutionizing medical diagnostics by utilizing image classification techniques that analyze intricate and delicate medical images using advanced algorithms such as deep learning and machine learning. AI has demonstrated exceptional accuracy in identifying cancer and other diseases, sometimes even matching or surpassing the level of expertise possessed by humans. The integration of AI into imaging technology enhances accuracy, minimizes human error, and expedites the diagnostic process, ultimately benefiting patient outcomes. There are advanced models for highly accurate identification of mammograms and skin cancer in dermatological images. Nevertheless, there are still obstacles that hinder the widespread implementation of AI in clinical settings: the difficulty in understanding AI decisions, concerns about data privacy, and the requirement for large, high-quality datasets.

CHAPTER ONE

1. INTRODUCTION

1.1 INTRODUCTION

AI has revolutionized medical diagnostics, enabling precise, speedy, and early identification of diseases. Image classification—a branch of artificial intelligence—involves extracting meaning from various medical imaging modalities, such as x-rays, magnetic resonance imaging (MRI), computed tomography (CT) scans, and histopathological images. Instead of relying on traditional algorithms, deep learning algorithms, specifically convolutional neural networks (CNNs), are employed to extract and learn important features from images, reducing the workload on radiologists and pathologists for interpretation. Image classification using AI has consistently and efficiently provided support for diagnostics in regions where there is a scarcity of medical professionals. AI systems possess the capability to identify even the most subtle patterns, thereby enhancing the accuracy of diagnoses and early detection of diseases such as cancer, pneumonia, diabetic retinopathy, and cardiovascular diseases. Previous research has demonstrated that artificial intelligence (AI) models can accurately identify specific medical conditions, often matching the performance of expert radiologists and even surpassing them in breast cancer detection, pneumonia detection, and skin cancer detection. These advancements in diagnostic tests would improve clinical decision-making and have the potential to make healthcare more accessible by incorporating these tools into mobile apps and remote clinics. Worries about the integration of artificial intelligence (AI) concerning data privacy and confidentiality, model interpretability, bias in training data, and clinical validation. Today, research is ongoing to develop methodologies that enhance performance and design ethical, explainable, and generalizable artificial intelligence systems for real-world applications in healthcare.

1.2 PROBLEM STATEMENT

Over the past few years, technology has revolutionized the way we interact, perform our jobs, and engage in leisure activities. The advent of smartphones and social media has revolutionized communication, making it possible for us to have constant connectivity through battery-powered apps that are conveniently located in our pockets. Furthermore, remote work and virtual meetings have become increasingly prevalent, offering alternative methods for collaboration. As technology continues to advance, society will experience both positive and negative impacts, along with new challenges and improvements.

1.3 OBJECTIVES

1. Analyze the latest advancements in artificial intelligence (AI) techniques for medical image classification, including the detection of tumors, fractures, and other anomalies in radiographs, CT scans, and MRIs, while also addressing their limitations.

2. Convolutional neural networks (CNNs) and other deep learning architectures are employed to recognize the essential characteristics in a medical image and patterns that will result in highly precise diagnostic results.
3. Develop an AI-powered image classification model that efficiently and accurately classifies medical images into specific diagnostic categories.
4. The research evaluates the effectiveness of different deep learning models, such as residual network (ResNet), visual geometry group (VGG), DenseNet, and inception, in order to identify the most suitable architecture for practical use in clinical settings.
5. The study investigates the ethical, regulatory, and clinical considerations of integrating artificial intelligence (ai)-based image classification tools into diagnostic workflows, with a focus on ensuring transparency, explainability, and patient safety.

CHAPTER TWO

2. METHODOLOGY

2.1 METHODOLOGY

The process of using artificial intelligence in medical diagnosis through image classification includes collecting data, preprocessing it, annotating it, selecting a model, training and validating it, testing and evaluating it, and finally deploying it for practical use. Obtaining labeled medical images from reliable sources is the initial step. Preprocessing improves image quality by applying normalization, resizing, and augmentation techniques. Expert-annotated data is employed for labeling and training. Deep learning models, such as convolutional neural networks (CNNs), are selected for extracting features and performing classification tasks. Training and validation are conducted on a training dataset, and the performance is evaluated using metrics such as accuracy, precision, recall, f1-score, and the area under the receiver operating characteristic curve (ROC - AUC).

2.2 STEPS OF METHODOLOGY

The process of using artificial intelligence for medical diagnosis through image classification typically follows these steps.

STEP 1: DATA COLLECTION AND PREPROCESSING:

Acquiring a medically verified dataset from reputable sources like nih chest x-ray dataset, isic skin cancer dataset, or Kaggle's diabetic retinopathy dataset is the first step, containing labeled medical images like x-rays, CT scans, MRIs, and dermatological images.

Preprocessing involves:

- The process entails adjusting the values of the image pixels to a standard scale.
- The process entails resizing images to a consistent input shape, such as 224x224.
- The process involves enhancing the dataset size and enhancing model generalization through techniques like rotation, flipping, and zooming.
- The process of noise removal can be achieved through techniques such as Gaussian filtering.
- The process of segmentation can be used to identify specific areas of interest, such as tumors or lesions.

STEP 2: FEATURE EXTRACTION

Deep learning models, particularly Convolutional Neural Networks (CNNs), are used to extract spatial features from medical images, often fine-tuned using transfer learning. The focus is on learning edge features, texture patterns, and localized abnormalities.

STEP 3: MODEL TRAINING AND VALIDATION

The subsequent phase involves training the neural network model on the prepared dataset after feature extraction.

- **Split ratio:**
70% of the training is done, followed by 15% validation and 15% testing.
- **Loss function:**
The binary cross entropy or categorical cross entropy is determined by the output labels.
- **Optimizer:**
Adam or SGD can be used with learning rate scheduling.
- **Evaluation Metrics:**
It provides information on accuracy, precision, recall, F1-score, and AUC-ROC Curve.
Cross-validation, such as k-fold, is employed to minimize overfitting and guarantee the model's broader applicability.

STEP 4: MODEL TESTING AND PERFORMANCE EVALUATION

The model is tested on an unseen dataset and its performance is assessed using a confusion matrix and ROC curves:

- Sensitivity refers to the true positive rate of an input.
 - The term "specificity" refers to the degree to which a statement accurately represents the true negative rate.
 - The overall diagnostic accuracy is a crucial aspect of medical diagnosis.
- The model decisions are validated through comparison with expert medical diagnostics or consensus labels.

STEP 5: MODEL DEPLOYMENT AND CLINICAL INTEGRATION

The model is validated and then integrated into a clinical decision support system (CDSS) during the validation phase:

- The model will be deployed on a healthcare platform that is either cloud-based or on-premises.
- The project involves creating a user-friendly interface (GUI) for radiologists and physicians.
- Real-time inference and reporting are crucial in ensuring accurate and timely information.
- The task involves ensuring compliance with the laws governing data protection, specifically HIPAA/GDPR.

STEP 6: ETHICAL CONSIDERATION AND BIAS MITIGATION

The ethical use of AI in medical image classification involves:

- The goal is to eliminate any potential bias in the training data.
- The goal is to promote transparency in decision-making through Explainable AI, such as Grad-CAM.
- The data includes information on a wide range of ethnicities and demographics.
- The focus is on fostering human-AI collaboration rather than replacing each other.

CHAPTER THREE

3. HISTORY OF MEDICAL DIAGNOSIS

3.1 EARLY FOUNDATION(1950s-1070s)

AI in medicine, a conceptual effort, originated from Alan Turing's theoretical work in the 1950s, but its applications were limited due to computing power limitations.

- **Early Expert Systems:**

In the 1970s, rule-based expert systems like MYCIN were developed to diagnose infectious blood diseases, providing a foundational logic for clinical decision-making.

3.2 INTRODUCTION OF MEDICAL IMAGE ANALYSIS (1980s-1990s)

In the 1980s, medical imaging advancements like CT and MRI led to increased demand for automated image analysis, requiring researchers to apply digital processing techniques.

- **EARLY ALGORITHM:**

The basic pattern recognition method utilized is edge detection, segmentation, and template matching.

- **LIMITATIONS:**

The early systems were rigid and had limited learning capabilities.

3.3 RISE OF MACHINE LEARNING(1990s-2010s)

Machine learning, specifically SVM and Random Forests, has revolutionized AI systems by enabling them to learn from data rather than relying solely on hand-crafted rules.

- **APPLICATIONS:**

Machine learning was utilized for classifying mammograms, detecting lung nodules in CT scans, and detecting diabetic retinopathy from fundus images.

- **LIMITATIONS:**

The performance of the system was hindered by the limitations imposed by feature engineering and the use of smaller datasets.

3.4 DEEP LEARNING REVOLUTION(2012-PRESENT)

In 2012, the success of deep convolutional neural networks (CNNs) on ImageNet led to their adoption in medical image classification.

- **KEY MILESTONES:**

- **2016:**

Google's DeepMind has developed an AI that accurately diagnoses diabetic retinopathy using retinal fundus images, demonstrating expert-level performance.

- **2017-2018:**

Stanford researchers utilized CNNs for skin cancer classification, achieving dermatologists' level accuracy.

- **Radiology:**

AI models have significantly outperformed radiologists in detecting pneumonia, COVID-19, and breast cancer using X-rays and MRIs.

3.5 RECENT TRENDS AND FUTURE OUTLOOK(2020s-PRESENT)

The medical imaging industry is transitioning towards explainable AI (XAI), multi-modal models, and federated learning to enhance trust, performance, and privacy.

- **MULTI-MODAL LEARNING:**

The integration of image data with electronic health records (EHRs) is a significant advancement in healthcare technology.

- **SELF-SUPERVISED LEARNING:**

The goal is to decrease the reliance on labeled datasets.

- **FDA APPROVALS:**

Over 100 AI-based medical imaging systems have been FDA-cleared for clinical use.

CHAPTER FOUR

4. CODE OF AI FOR MEDICAL DIAGNOSIS

4.1 CODE OF AI FOR MEDICAL DIAGNOSIS

```
import os
import cv2
import numpy as np
import matplotlib.pyplot as plt
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Flatten, Conv2D, MaxPooling2D, Dropout,
BatchNormalization
from tensorflow.keras.utils import to_categorical
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report, confusion_matrix
import heapq
# ----- Load and Preprocess Images -----
def load_images_from_folder(folder, img_size=64):
    """
    Load images from a folder structure where each subfolder represents a class
    Returns images, labels, and a mapping of class names to numeric labels
    """
    images = []
    labels = []
    label_map = {} # String labels to numbers
    label_counter = 0
    # Check if folder exists
    if not os.path.exists(folder):
        print(f'Error: Folder '{folder}' does not exist!')
```

```

        return np.array([]), np.array([]), {}
    for label_name in os.listdir(folder):
        label_folder = os.path.join(folder, label_name)
        if os.path.isdir(label_folder):
            if label_name not in label_map:
                label_map[label_name] = label_counter
                label_counter += 1
            print(f'Loading images from class: {label_name}')
            file_count = 0
            for filename in os.listdir(label_folder):
                img_path = os.path.join(label_folder, filename)
                try:
                    img = cv2.imread(img_path, cv2.IMREAD_GRAYSCALE)
                    if img is not None:
                        img = cv2.resize(img, (img_size, img_size))
                        images.append(img)
                        labels.append(label_map[label_name])
                        file_count += 1
                except Exception as e:
                    print(f'Error loading {img_path}: {e}')
            print(f' - Loaded {file_count} images')
    # Create reverse mapping (numeric to string)
    reverse_label_map = {v: k for k, v in label_map.items()}
    print(f'Total images loaded: {len(images)}')
    print(f'Classes found: {list(label_map.keys())}')
    return np.array(images), np.array(labels), label_map, reverse_label_map

# ----- ANN Model -----

def build_ann(input_shape, num_classes):

```

```

"""Build a simple Artificial Neural Network"""
model = Sequential([
    Flatten(input_shape=input_shape),
    Dense(128, activation='relu'),
    Dropout(0.2), # Add dropout for regularization
    Dense(64, activation='relu'),
    Dense(num_classes, activation='softmax')
])
model.summary()
return model

# ----- CNN Model -----
def build_cnn(input_shape, num_classes):
    """Build a Convolutional Neural Network"""
    model = Sequential([
        Conv2D(32, (3,3), activation='relu', padding='same', input_shape=input_shape),
        MaxPooling2D(pool_size=(2,2)),
        Conv2D(64, (3,3), activation='relu', padding='same'),
        MaxPooling2D(pool_size=(2,2)),
        Conv2D(128, (3,3), activation='relu', padding='same'),
        MaxPooling2D(pool_size=(2,2)),
        Flatten(),
        Dense(128, activation='relu'),
        Dropout(0.3),
        Dense(num_classes, activation='softmax')
    ])
    model.summary()
    return model

# ----- BNN (Simulated via Binary Activation) -----

```

```

def build_bnn(input_shape, num_classes):
    """
    Build a network that simulates some aspects of Binary Neural Networks
    Note: This is not a true BNN as it doesn't use binary weights
    """
    model = Sequential([
        Flatten(input_shape=input_shape),
        Dense(128, activation='relu'),
        BatchNormalization(),
        Dense(64, activation='relu'),
        BatchNormalization(),
        Dense(num_classes, activation='softmax')
    ])
    model.summary()
    return model

# ----- Graph Search Algorithms -----

def dfs(graph, start, goal, visited=None):
    """Depth-First Search algorithm for graph traversal"""
    if visited is None:
        visited = set()
    visited.add(start)
    if start == goal:
        return [start]
    for neighbor in graph.get(start, []):
        if neighbor not in visited:
            path = dfs(graph, neighbor, goal, visited)
            if path:
                return [start] + path

```

```

    return None

def bfs(graph, start, goal):
    """Breadth-First Search algorithm for graph traversal"""
    visited = set()
    queue = [[start]]
    if start == goal:
        return [start]
    while queue:
        path = queue.pop(0)
        node = path[-1]
        if node == goal:
            return path
        elif node not in visited:
            for neighbor in graph.get(node, []):
                new_path = list(path)
                new_path.append(neighbor)
                queue.append(new_path)
            visited.add(node)
    return None

def best_first_search(graph, start, goal, heuristic):
    """Best-First Search algorithm using a heuristic function"""
    visited = set()
    heap = [(heuristic.get(start, float('inf')), [start])]
    while heap:
        _, path = heapq.heappop(heap)
        node = path[-1]
        if node == goal:
            return path

```

```

    if node not in visited:
        visited.add(node)
        for neighbor in graph.get(node, []):
            if neighbor not in visited:
                new_path = path + [neighbor]
                heapq.heappush(heap, (heuristic.get(neighbor, float('inf')), new_path))
    return None

def beam_search(graph, start, goal, heuristic, beam_width=2):
    """Beam Search algorithm with specified beam width"""
    if start == goal:
        return [start]
    queue = [[start]]
    visited = set([start])
    while queue:
        # Sort by heuristic value and keep only up to beam_width paths
        queue.sort(key=lambda path: heuristic.get(path[-1], float('inf')))
        queue = queue[:beam_width]
        next_queue = []
        for path in queue:
            node = path[-1]
            if node == goal:
                return path
            for neighbor in graph.get(node, []):
                if neighbor not in visited:
                    visited.add(neighbor)
                    new_path = list(path)
                    new_path.append(neighbor)
                    next_queue.append(new_path)

```



```

        queue = next_queue

    if not queue: # If queue is empty, no path exists
        break

    return None

# ----- Evaluation Function -----

def evaluate_model(model, x_test, y_test, class_names=None):
    """Evaluate model performance and display metrics and visualizations"""

    # Get predictions
    y_pred_prob = model.predict(x_test)
    y_pred_classes = np.argmax(y_pred_prob, axis=1)
    y_test_classes = np.argmax(y_test, axis=1)

    # Print classification report
    if class_names:
        target_names = [class_names.get(i, f"Class {i}") for i in
range(len(np.unique(y_test_classes)))]

        print("\nClassification Report:")

        print(classification_report(y_test_classes, y_pred_classes, target_names=target_names))
    else:
        print("\nClassification Report:")

        print(classification_report(y_test_classes, y_pred_classes))

    # Print confusion matrix
    cm = confusion_matrix(y_test_classes, y_pred_classes)

    print("\nConfusion Matrix:")

    print(cm)

    # Plot confusion matrix as heatmap
    plt.figure(figsize=(10, 8))
    plt.imshow(cm, interpolation='nearest', cmap=plt.cm.Blues)
    plt.title('Confusion Matrix')
    plt.colorbar()

```

```

if class_names:
    tick_marks = np.arange(len(np.unique(y_test_classes)))
    class_labels = [class_names.get(i, f"Class {i}") for i in
range(len(np.unique(y_test_classes)))]
    plt.xticks(tick_marks, class_labels, rotation=45)
    plt.yticks(tick_marks, class_labels)
plt.tight_layout()
plt.ylabel('True Label')
plt.xlabel('Predicted Label')
plt.show()
# Plot sample misclassifications
misclassified = np.where(y_pred_classes != y_test_classes)[0]
if len(misclassified) > 0:
    n_display = min(5, len(misclassified))
    plt.figure(figsize=(15, 3))
    for i, idx in enumerate(misclassified[:n_display]):
        plt.subplot(1, n_display, i+1)
        # If grayscale image
        if x_test[idx].shape[-1] == 1:
            plt.imshow(x_test[idx].reshape(x_test[idx].shape[0], x_test[idx].shape[1]),
cmap='gray')
        else:
            plt.imshow(x_test[idx])
        true_class = y_test_classes[idx]
        pred_class = y_pred_classes[idx]

    if class_names:
        true_label = class_names.get(true_class, f"Class {true_class}")
        pred_label = class_names.get(pred_class, f"Class {pred_class}")

```

```

        plt.title(f'True: {true_label}\nPred: {pred_label}')
    else:
        plt.title(f'True: {true_class}\nPred: {pred_class}')
    plt.axis('off')
    plt.tight_layout()
    plt.show()

    return y_pred_classes, y_test_classes

# ----- Create Sample Dataset -----

def create_sample_dataset(output_folder='sample_dataset', num_classes=3,
                           samples_per_class=50, img_size=64):
    """Create a sample dataset for demonstration purposes"""
    if not os.path.exists(output_folder):
        os.makedirs(output_folder)
    shapes = ['circle', 'square', 'triangle']
    for i in range(min(num_classes, len(shapes))):
        class_folder = os.path.join(output_folder, shapes[i])
        if not os.path.exists(class_folder):
            os.makedirs(class_folder)
        print(f'Creating {samples_per_class} {shapes[i]} images...')
        for j in range(samples_per_class):
            # Create a blank image
            img = np.zeros((img_size, img_size), dtype=np.uint8)
            # Add some randomness to position and size
            center_x = np.random.randint(img_size//4, 3*img_size//4)
            center_y = np.random.randint(img_size//4, 3*img_size//4)
            size = np.random.randint(img_size//4, img_size//2)
            # Draw shape based on class
            if shapes[i] == 'circle':
                cv2.circle(img, (center_x, center_y), size//2, 255, -1)

```

```

elif shapes[i] == 'square':
    x1 = max(0, center_x - size//2)
    y1 = max(0, center_y - size//2)
    x2 = min(img_size-1, center_x + size//2)
    y2 = min(img_size-1, center_y + size//2)
    cv2.rectangle(img, (x1, y1), (x2, y2), 255, -1)
elif shapes[i] == 'triangle':
    points = np.array([
        [center_x, center_y - size//2],
        [center_x - size//2, center_y + size//2],
        [center_x + size//2, center_y + size//2]
    ])
    cv2.fillPoly(img, [points], 255)
# Add some noise
noise = np.random.normal(0, 15, img.shape).astype(np.uint8)
img = cv2.add(img, noise)
# Save image
cv2.imwrite(os.path.join(class_folder, f'{shapes[i]}_{j}.png'), img)
print(f'Sample dataset created at {output_folder}')
return output_folder
# ----- Main Function -----
def main():
    # 1. Create sample dataset if needed
    use_sample_data = True
    if use_sample_data:
        image_folder = create_sample_dataset(num_classes=3, samples_per_class=100)
    else:
        # Use your own dataset path here

```

```

    image_folder = 'path_to_your_dataset'

# 2. Load Dataset

images, labels, label_map, reverse_label_map = load_images_from_folder(image_folder)

if len(images) == 0:
    print("No images loaded. Exiting...")
    return

# Normalize images
images = images / 255.0

# Add channel dimension for grayscale images
images = np.expand_dims(images, axis=-1)

# One-hot encode labels
labels_categorical = to_categorical(labels)

# Split dataset
x_train, x_test, y_train, y_test = train_test_split(images, labels_categorical, test_size=0.2,
random_state=42)

print(f"\nDataset split: {x_train.shape[0]} training samples, {x_test.shape[0]} test samples")
print(f"Image shape: {x_train.shape[1:]}")
input_shape = x_train.shape[1:]
num_classes = y_train.shape[1]

# 3. Run neural network training and evaluation
run_neural_networks = True

if run_neural_networks:
    # Build and Train ANN
    print("\n" + "="*50)
    print("TRAINING ARTIFICIAL NEURAL NETWORK (ANN)")
    print("="*50)
    ann = build_ann(input_shape, num_classes)
    ann.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy'])
    ann_history = ann.fit(

```

```

    x_train, y_train,
    epochs=10,
    batch_size=32,
    validation_data=(x_test, y_test),
    verbose=1
)
print("\n" + "="*50)
print("ANN EVALUATION")
print("="*50)
ann_results = evaluate_model(ann, x_test, y_test, reverse_label_map)
# Plot training history
plt.figure(figsize=(12, 4))
plt.subplot(1, 2, 1)
plt.plot(ann_history.history['accuracy'], label='Train Accuracy')
plt.plot(ann_history.history['val_accuracy'], label='Validation Accuracy')
plt.title('ANN Accuracy')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.legend()
plt.subplot(1, 2, 2)
plt.plot(ann_history.history['loss'], label='Train Loss')
plt.plot(ann_history.history['val_loss'], label='Validation Loss')
plt.title('ANN Loss')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.legend()
plt.tight_layout()
plt.show()

```

```

# Build and Train CNN

print("\n" + "="*50)
print("TRAINING CONVOLUTIONAL NEURAL NETWORK (CNN)")
print("="*50)

cnn = build_cnn(input_shape, num_classes)
cnn.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy'])
cnn_history = cnn.fit(
    x_train, y_train,
    epochs=10,
    batch_size=32,
    validation_data=(x_test, y_test),
    verbose=1
)

print("\n" + "="*50)
print("CNN EVALUATION")
print("="*50)

cnn_results = evaluate_model(cnn, x_test, y_test, reverse_label_map)

# Plot training history
plt.figure(figsize=(12, 4))
plt.subplot(1, 2, 1)
plt.plot(cnn_history.history['accuracy'], label='Train Accuracy')
plt.plot(cnn_history.history['val_accuracy'], label='Validation Accuracy')
plt.title('CNN Accuracy')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.legend()

plt.subplot(1, 2, 2)
plt.plot(cnn_history.history['loss'], label='Train Loss')

```

```

plt.plot(cnn_history.history['val_loss'], label='Validation Loss')
plt.title('CNN Loss')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.legend()
plt.tight_layout()
plt.show()

# Build and Train BNN (simulated)
print("\n" + "="*50)
print("TRAINING BINARY NEURAL NETWORK (BNN SIMULATION)")
print("="*50)

bnn = build_bnn(input_shape, num_classes)
bnn.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy'])
bnn_history = bnn.fit(
    x_train, y_train,
    epochs=10,
    batch_size=32,
    validation_data=(x_test, y_test),
    verbose=1
)
print("\n" + "="*50)
print("BNN EVALUATION")
print("="*50)

bnn_results = evaluate_model(bnn, x_test, y_test, reverse_label_map)

# Plot training history
plt.figure(figsize=(12, 4))
plt.subplot(1, 2, 1)
plt.plot(bnn_history.history['accuracy'], label='Train Accuracy')

```



```

plt.plot(bnn_history.history['val_accuracy'], label='Validation Accuracy')
plt.title('BNN Accuracy')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.legend()

plt.subplot(1, 2, 2)
plt.plot(bnn_history.history['loss'], label='Train Loss')
plt.plot(bnn_history.history['val_loss'], label='Validation Loss')
plt.title('BNN Loss')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.legend()
plt.tight_layout()
plt.show()

# Compare models
plt.figure(figsize=(12, 5))
plt.subplot(1, 2, 1)
plt.plot(ann_history.history['val_accuracy'], label='ANN')
plt.plot(cnn_history.history['val_accuracy'], label='CNN')
plt.plot(bnn_history.history['val_accuracy'], label='BNN')
plt.title('Model Comparison - Validation Accuracy')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.legend()

plt.subplot(1, 2, 2)
plt.plot(ann_history.history['val_loss'], label='ANN')
plt.plot(cnn_history.history['val_loss'], label='CNN')
plt.plot(bnn_history.history['val_loss'], label='BNN')

```

```

plt.title('Model Comparison - Validation Loss')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.legend()
plt.tight_layout()
plt.show()

# 4. Graph Algorithms (Example for Decision Flow)

run_graph_algorithms = True

if run_graph_algorithms:
    print("\n" + "="*50)
    print("GRAPH SEARCH ALGORITHMS")
    print("="*50)
    # Define a sample graph
    graph = {
        'Start': ['A', 'B'],
        'A': ['C', 'D'],
        'B': ['E'],
        'C': ['F'],
        'D': [],
        'E': ['G'],
        'F': [],
        'G': ['Goal'],
        'Goal': []
    }
    # Heuristic values (estimated distance to goal)
    heuristic = {'Start': 6, 'A': 4, 'B': 3, 'C': 3, 'D': 4, 'E': 2, 'F': 2, 'G': 1, 'Goal': 0}
    # Visualize the graph
    plt.figure(figsize=(10, 6))

```

```

# Node positions (manually defined for this example)
positions = {
    'Start': (1, 4),
    'A': (3, 6),
    'B': (3, 2),
    'C': (5, 7),
    'D': (5, 5),
    'E': (5, 2),
    'F': (7, 7),
    'G': (7, 2),
    'Goal': (9, 4)
}

# Draw nodes
for node, pos in positions.items():
    plt.plot(pos[0], pos[1], 'o', markersize=20, color='lightblue')
    plt.text(pos[0], pos[1], f"{node}\n(h={heuristic.get(node, 'N/A')})",
             ha='center', va='center', fontweight='bold')

# Draw edges
for node, neighbors in graph.items():
    for neighbor in neighbors:
        plt.plot([positions[node][0], positions[neighbor][0]],
                 [positions[node][1], positions[neighbor][1]], 'k-', alpha=0.6)

plt.title("Example Graph with Heuristic Values")
plt.axis('off')
plt.tight_layout()
plt.show()

# Execute graph search algorithms
dfs_path = dfs(graph, 'Start', 'Goal')

```

```

bfs_path = bfs(graph, 'Start', 'Goal')
best_first_path = best_first_search(graph, 'Start', 'Goal', heuristic)
beam_path = beam_search(graph, 'Start', 'Goal', heuristic, beam_width=2)
# Print results
print("\nGraph Search Results:")
print(f'DFS Path: {dfs_path}')
print(f'BFS Path: {bfs_path}')
print(f'Best-First Search Path: {best_first_path}')
print(f'Beam Search Path (width=2): {beam_path}')
# Visualize search paths
algorithms = {
    'DFS': dfs_path,
    'BFS': bfs_path,
    'Best-First Search': best_first_path,
    'Beam Search (width=2)': beam_path
}
plt.figure(figsize=(15, 10))
for i, (name, path) in enumerate(algorithms.items(), 1):
    plt.subplot(2, 2, i)
    # Draw nodes
    for node, pos in positions.items():
        if node in path:
            color = 'lightgreen'
            if node == 'Start':
                color = 'green'
            elif node == 'Goal':
                color = 'red'
        else:

```

```

        color = 'lightblue'
    plt.plot(pos[0], pos[1], 'o', markersize=20, color=color)
    plt.text(pos[0], pos[1], node, ha='center', va='center', fontweight='bold')
# Draw edges
for node, neighbors in graph.items():
    for neighbor in neighbors:
        if node in path and neighbor in path and path.index(neighbor) == path.index(node) +
1:
            plt.plot([positions[node][0], positions[neighbor][0]],
                    [positions[node][1], positions[neighbor][1]], 'r-', linewidth=2.5)
        else:
            plt.plot([positions[node][0], positions[neighbor][0]],
                    [positions[node][1], positions[neighbor][1]], 'k-', alpha=0.3)
    plt.title(f'{name}: {' → '.join(path) if path else 'No path found'}')
    plt.axis('off')
    plt.tight_layout()
    plt.show()
if __name__ == '__main__':
    main()

```

CHAPTER FIVE

5. RESULTS

5.1 RESULTS OF AI MEDICAL DIAGNOSIS FOR IMAGE CLASSIFICATION

```
2025-05-20 09:56:14.551222: I tensorflow/core/util/port.cc:153] oneDNN custom operations are on. You may see slightly different numerical results due to floating-point round-off errors from different computation orders. To turn them off, set the environment variable 'TF_ENABLE_ONEDNN_OPTS=0'.
2025-05-20 09:56:19.300605: I tensorflow/core/util/port.cc:153] oneDNN custom operations are on. You may see slightly different numerical results due to floating-point round-off errors from different computation orders. To turn them off, set the environment variable 'TF_ENABLE_ONEDNN_OPTS=0'.
Creating 100 circle images...
Creating 100 square images...
Creating 100 triangle images...
Sample dataset created at sample_dataset
Loading images from class: circle
- Loaded 100 images
Loading images from class: square
- Loaded 100 images
Loading images from class: triangle
- Loaded 100 images
Total images loaded: 300
Classes found: ['circle', 'square', 'triangle']

Dataset split: 240 training samples, 60 test samples
Image shape: (64, 64, 1)
```

Figure 5.1:REUSLTS OF TENSOR FLOW

```
=====
TRAINING ARTIFICIAL NEURAL NETWORK (ANN)
=====
D:\ANAACONDA\Lib\site-packages\Keras\src\layers\reshaping\flatten.py:37: UserWarning: Do not pass an 'input_shape'/'input_dim' argument to a layer. When using Sequential models, prefer using an 'Input(shape)' object as the first layer in the model instead.
  super().__init__(**kwargs)
2025-05-20 09:56:33.246436: I tensorflow/core/platform/cpu_feature_guard.cc:210] This TensorFlow binary is optimized to use available CPU instructions in performance-critical operations.
To enable the following instructions: AVX2 FMA, in other operations, rebuild TensorFlow with the appropriate compiler flags.
Model: "sequential"


```

Layer (type)	Output Shape	Param #
flatten (Flatten)	(None, 4096)	0
dense (Dense)	(None, 128)	524,416
dropout (Dropout)	(None, 128)	0
dense_1 (Dense)	(None, 64)	8,256
dense_2 (Dense)	(None, 3)	195

```

Total params: 532,867 (2.03 MB)
Trainable params: 532,867 (2.03 MB)
Non-trainable params: 0 (0.00 B)

Epoch 1/10
8/8 2s 92ms/step - accuracy: 0.3482 - loss: 1.3994 - val_accuracy: 0.3667 - val_loss: 1.1422
Epoch 2/10
8/8 0s 25ms/step - accuracy: 0.4852 - loss: 1.0895 - val_accuracy: 0.3667 - val_loss: 1.1223
Epoch 3/10
8/8 0s 25ms/step - accuracy: 0.4945 - loss: 1.0773 - val_accuracy: 0.3833 - val_loss: 1.1051
Epoch 4/10
8/8 0s 24ms/step - accuracy: 0.6674 - loss: 0.8200 - val_accuracy: 0.4000 - val_loss: 1.1410
Epoch 5/10
8/8 0s 23ms/step - accuracy: 0.6915 - loss: 0.7192 - val_accuracy: 0.3333 - val_loss: 1.3100
Epoch 6/10
8/8 0s 23ms/step - accuracy: 0.7956 - loss: 0.5688 - val_accuracy: 0.4833 - val_loss: 1.1041
Epoch 7/10
8/8 0s 24ms/step - accuracy: 0.8512 - loss: 0.4385 - val_accuracy: 0.4000 - val_loss: 1.1464
Epoch 8/10
8/8 0s 29ms/step - accuracy: 0.9018 - loss: 0.3287 - val_accuracy: 0.4667 - val_loss: 1.0144
Epoch 9/10
8/8 0s 29ms/step - accuracy: 0.9894 - loss: 0.2078 - val_accuracy: 0.4667 - val_loss: 1.0310
Epoch 10/10
8/8 0s 26ms/step - accuracy: 0.9801 - loss: 0.1647 - val_accuracy: 0.4500 - val_loss: 1.1788
```

Figure 5.2:RESULTS OF TRAINING ARTIFICIAL NEURAL NETWORK

```

=====
ANN EVALUATION
=====
2/2 ████████████████████ 0s 91ms/step

Classification Report:

```

	precision	recall	f1-score	support
circle	0.41	0.64	0.50	22
square	0.38	0.50	0.43	16
triangle	1.00	0.23	0.37	22
accuracy			0.45	60
macro avg	0.60	0.45	0.43	60
weighted avg	0.62	0.45	0.43	60

```

Confusion Matrix:
[[14  8  0]
 [ 8  8  0]
 [12  5  5]]

```

Figure 5.3:RESULTS OF ANN EVALUATION

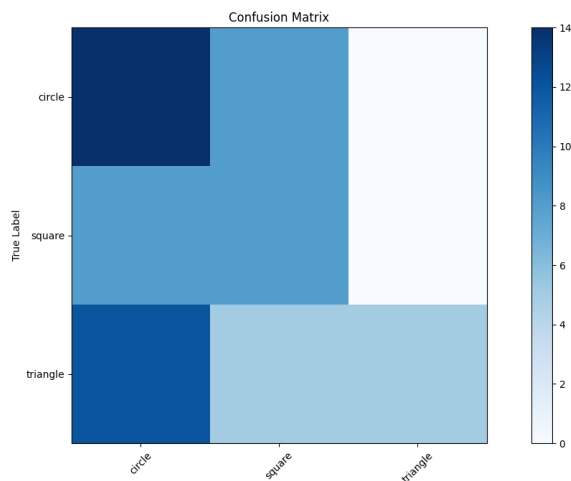


Figure 5.4:GRAPH OF CONFUSION MATRIX OF ANN EVALUATION

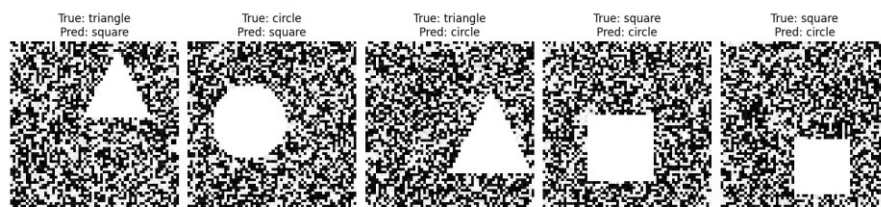


Figure 5.5:CLASSIFICATION OF MEDICAL DIAGNOSIS OF SHAPES OF ANN EVALUATION

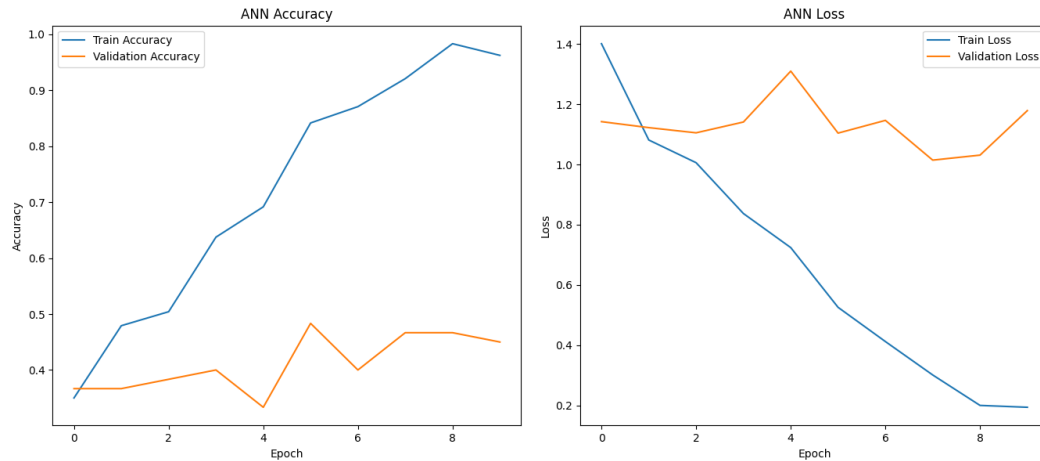


Figure 5.6:GRAPHS OF ANN ACCURACY AND ANN LOSS OF ANN EVALUATION

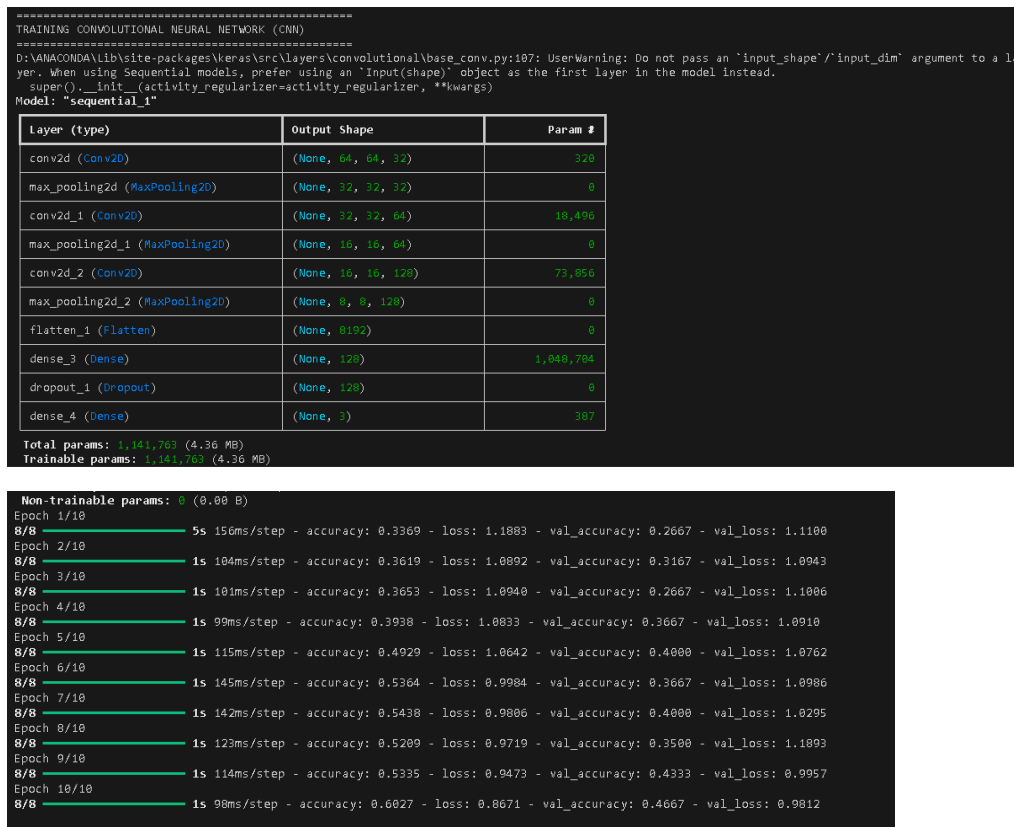


Figure 5.7: RESULTS OF TRAINING CONVOLUTIONAL NEURAL NETWORK


```

=====
CNN EVALUATION
=====
2/2 ██████████ 0s 156ms/step

Classification Report:
      precision    recall  f1-score   support

   circle      0.27      0.18      0.22         22
   square      0.67      0.50      0.57         16
  triangle      0.48      0.73      0.58         22

   accuracy      0.47              0.47         60
  macro avg      0.47      0.47      0.46         60
 weighted avg      0.45      0.47      0.44         60

Confusion Matrix:
[[ 4  4 14]
 [ 5  8  3]
 [ 6  0 16]]

```

Figure 5.8: RESULTS OF CNN EVALUATION

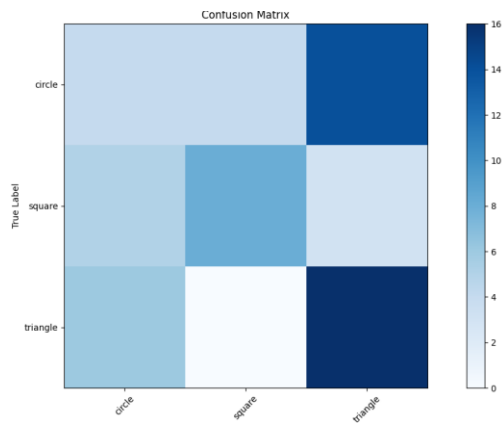


Figure 5.9: GRPAH OF CONFUSION MARTIX OF CNN EVOLUTION

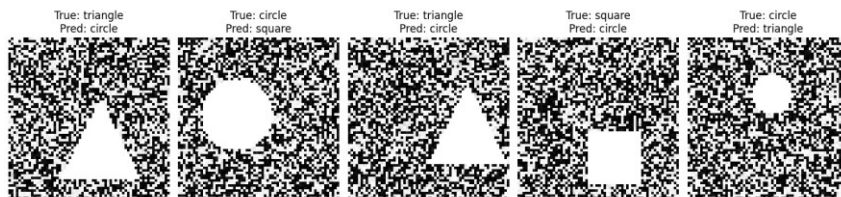


Figure 5.10: CLASSIFICATION OF MEDICAL DIAGNOSIS OF SHAPES OF CNN EVALUATION

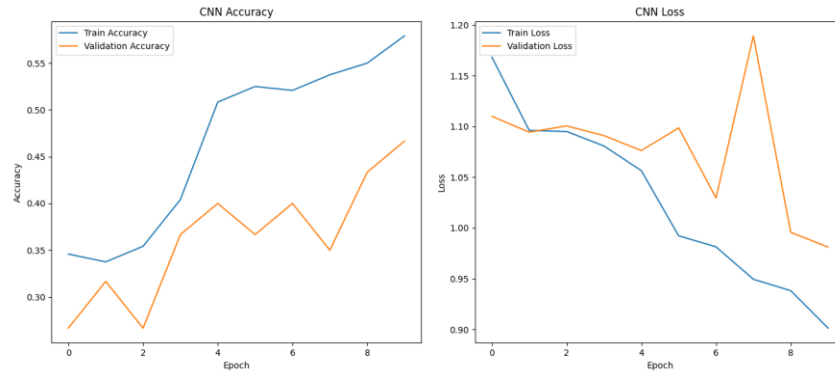


Figure 5.11: GRAPHS OF CNN ACCURACY AND CNN LOSS OF CNN EVALUATION

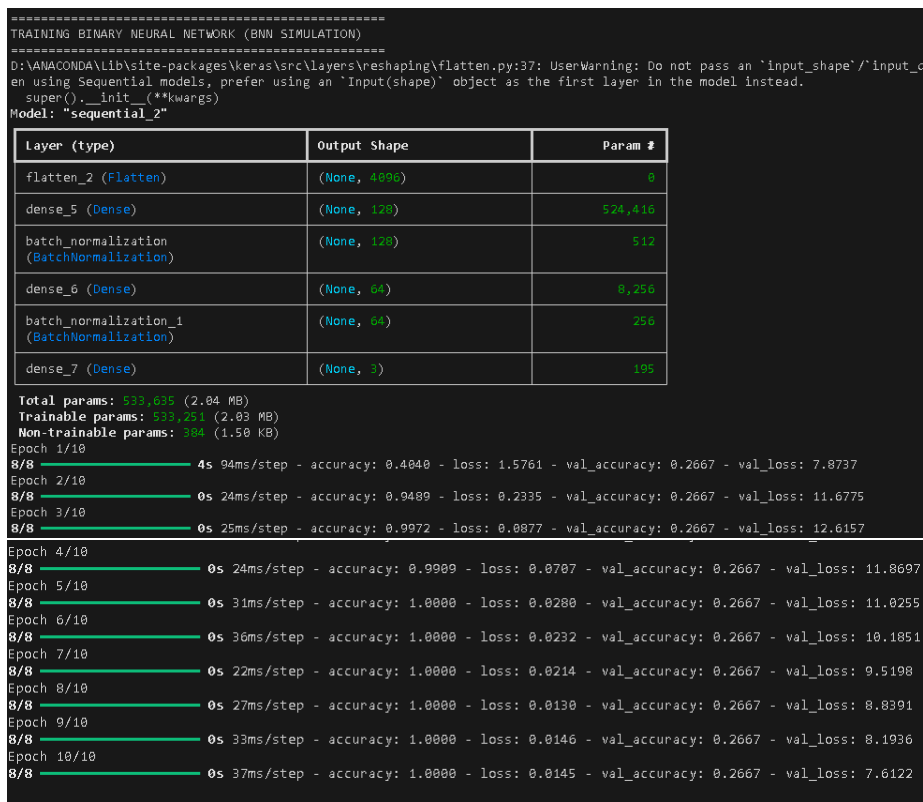


Figure 5.12: RESULTS OF TRAINING BINARY NEURAL NETWORK

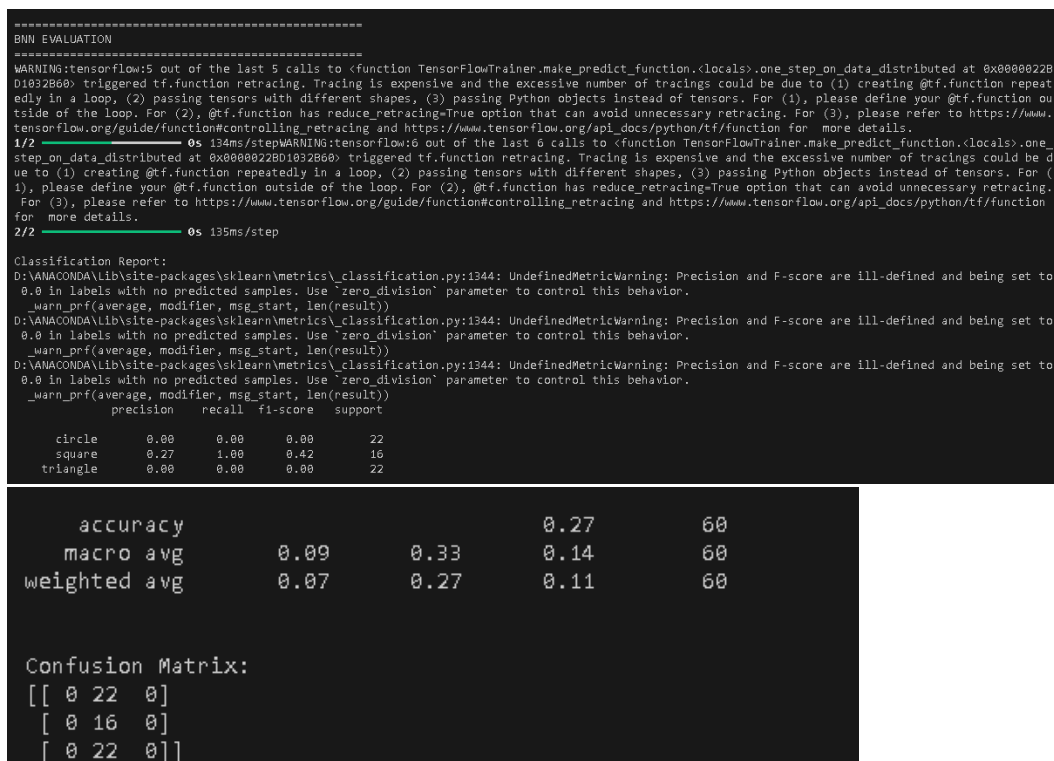


Figure 5.13:RESULT OF BNN EVALUTION

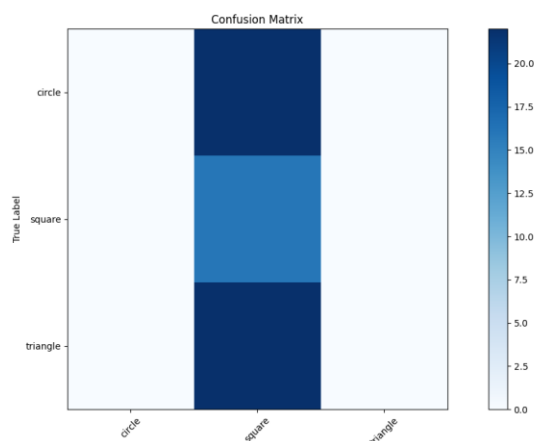


Figure 5.14: GRPAH OF CONFUSION MARTIX OF BNN EVOLUTION

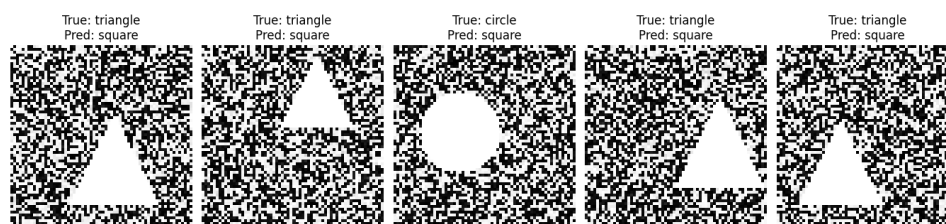


Figure 5.15: CLASSIFICATION OF MEDICAL DIAGNOSIS OF SHAPES OF BNN EVALUTION

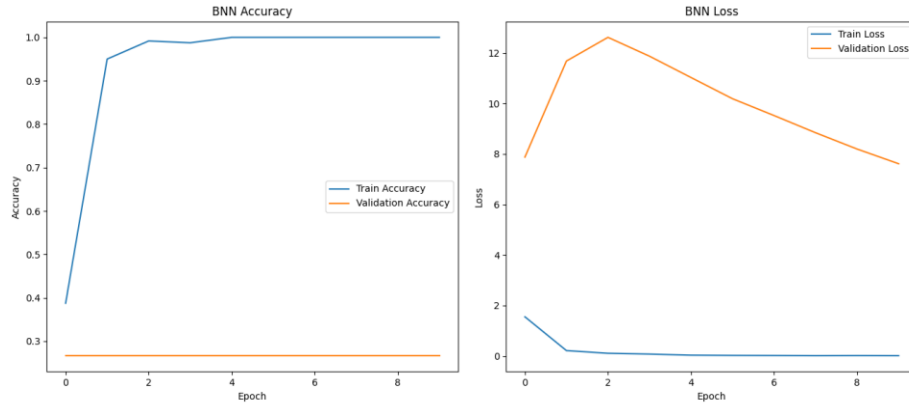


Figure 5.16: GRAPHS OF BNN ACCURACY AND BNN LOSS OF BNN EVALUATION

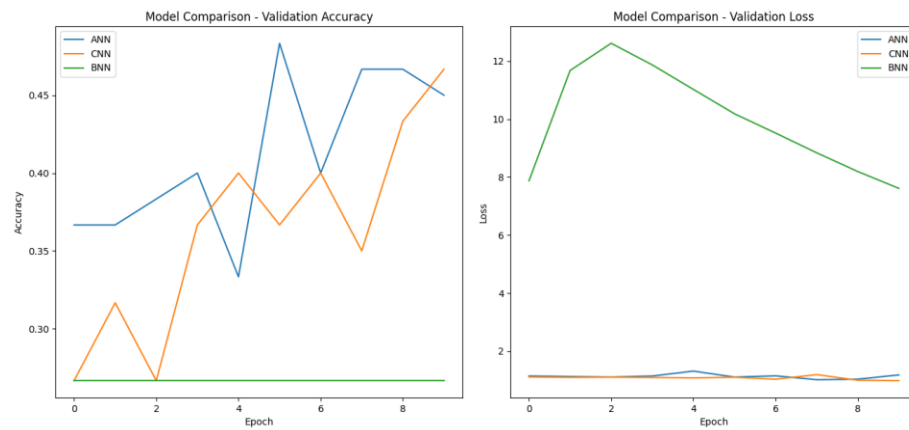


Figure 5.17: MODEL COMPARSION OF ANN, CNN AND BNN MODEL

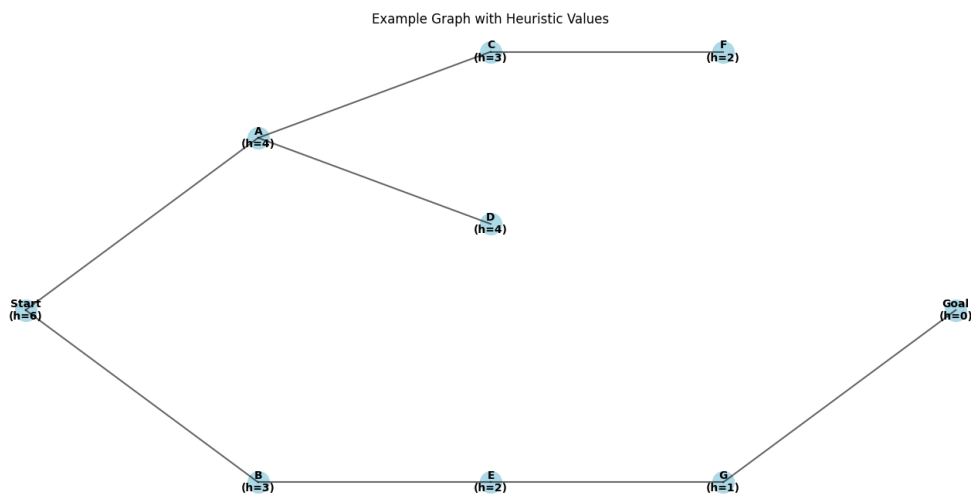


Figure 5.18: GRAPH OF BEAM SEARCH ALGORITHM

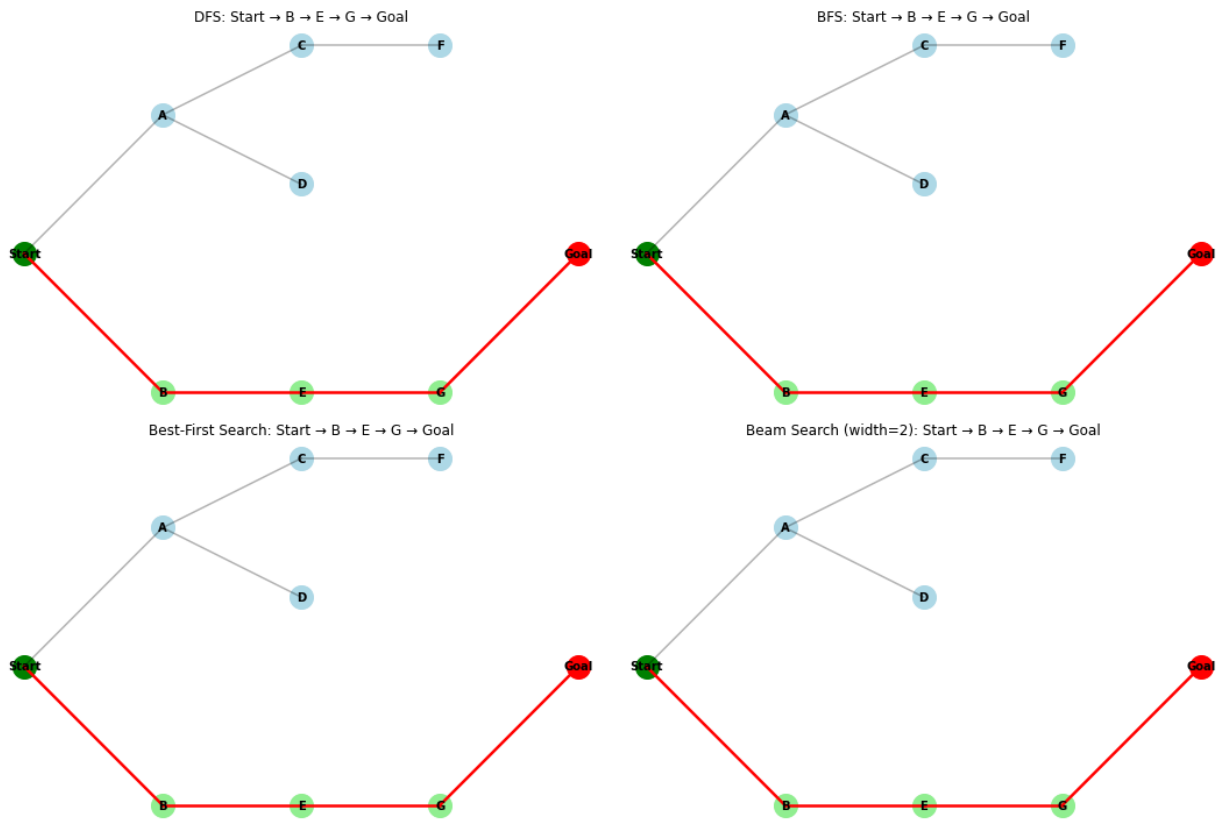


Figure 5.19:GRPAH OF BEST-FIRST SEARCH ALGORTHIM

CHAPTER SIX

6. CONCLUSION

6.1 CONCLUSION

In conclusion, we understand the AI FOR MEDICAL DIAGNOSIS the implemented the algorithms such as breadth first search, depth first search, best first search. The ARITFICAL NEURAL NETWORK,CONVOLUTION NEURAL NETWORK.WE IMPLEMENTED THIS CODE FOR THE AI FOR MEDICAL DIAGNOSIS.WE CHECK THE MEDICAL PROBLEMS for the health checkups, operations OF TUMORS ETC.

References

- [1] X. Wang, Y. Peng, L. Lu, Z. Lu, M. Bagheri, and R. M. Summers, "ChestX-ray8: Hospital-scale chest X-ray database and benchmarks on weakly-supervised classification and localization of common thorax diseases," *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2017, pp. 3462–3471.
- [2] P. Rajpurkar, J. Irvin, K. Zhu, et al., "CheXNet: Radiologist-level pneumonia detection on chest X-rays with deep learning," *arXiv preprint arXiv:1711.05225*, 2017.
- [3] A. Esteva, B. Kuprel, R. A. Novoa, et al., "Dermatologist-level classification of skin cancer with deep neural networks," *Nature*, vol. 542, no. 7639, pp. 115–118, 2017.
- [4] D. S. Kermany, M. Goldbaum, W. Cai, et al., "Identifying medical diagnoses and treatable diseases by image-based deep learning," *Cell*, vol. 172, no. 5, pp. 1122–1131, 2018.
- [5] E. Topol, *Deep Medicine: How Artificial Intelligence Can Make Healthcare Human Again*, Basic Books, 2019.
- [6] J. Amann, A. Blasimme, E. Vayena, D. Frey, and V. I. Madai, "Explainability for artificial intelligence in healthcare: a multidisciplinary perspective," *BMC Med. Inform. Decis. Mak.*, vol. 20, 2020.